Guía: cómo subir tus cambios al repo

Para el grupo 4 · MCDI500. Tiempo de lectura: 5 minutos. Tiempo de hacerlo: 2 minutos una vez que le pillas el truco.

> **La regla de oro:** nadie sube directo a `main`. Cada cambio va en una **rama propia** y entra por un **Pull Request (PR)** que otro compañero revisa. No es burocracia: es para que nadie pise el trabajo de otro.

El flujo en 6 pasos

```
1. Bajar lo último  →  2. Crear tu rama  →  3. Trabajar  →  4. Guardar (commit)  →  5. Subir (push)  →  6. Abrir PR
```

> **¿Cuál opción elijo?** Sin experiencia con git → **A**. Cómodo con la terminal → **B**. Quieres que una IA haga los comandos → **C**. El resultado es idéntico: una rama y un PR.

Opción A · GitHub Desktop (la fácil, sin comandos)

Una sola vez: preparación

1. Instala **GitHub Desktop**: https://desktop.github.com
2. Inicia sesión con tu cuenta de GitHub (`File → Options → Accounts`).
3. `File → Clone repository` → pestaña **URL** → pega: `https://github.com/yetgx/mcdi500-tmdb-grupo4.git` → elige una carpeta → **Clone**.
4. Prepara el entorno (ver README, sección *Requisitos y ejecución*).

Cada vez que quieras subir algo

#	Qué hacer	Dónde
1	**Fetch origin** y luego **Pull origin**	Botón azul de arriba
2	**Current Branch** → **New Branch**, ponle nombre (ej. `feat/f2-imputacion`) y **Create Branch**	Barra superior

#	Qué hacer	Dónde
3	Trabaja normal: edita notebooks, agrega archivos	Tu editor / Jupyter
4	Vuelve a GitHub Desktop. A la izquierda verás los archivos cambiados. Revisa que **solo estén los que tú tocaste**. Escribe un mensaje abajo a la izquierda (ej. `feat: agrega imputacion de runtime por mediana`) y **Commit to feat/f2-imputacion**	Abajo a la izquierda
5	**Publish branch** (o **Push origin** si ya estaba publicada)	Botón azul de arriba
6	**Create Pull Request** (aparece en el mismo botón azul). Se abre el navegador: rellena la plantilla y pulsa **Create pull request**	Navegador

Listo. Avisa al grupo por WhatsApp con el enlace del PR.

Opción B · Terminal (Git Bash o PowerShell)

Una sola vez

```
git clone https://github.com/yetgx/mcdi500-tmdb-grupo4.git
cd mcdi500-tmdb-grupo4
git config user.name  "Tu Nombre"
git config user.email "tu-correo-de-github@ejemplo.com"
```

Cada vez

```
git switch main
git pull                                  # 1. bajar lo último
git switch -c feat/f2-imputacion          # 2. tu rama nueva

# ... trabajas ...

git status                                # ver qué cambió
git add F2/S2_F2_Limpieza.ipynb           # 4. agrega SOLO lo tuyo (evita "git add .")
git commit -m "feat: agrega imputacion de runtime por mediana"
git push -u origin feat/f2-imputacion     # 5. subir
```

Git te imprime un enlace en el terminal: ábrelo para crear el PR (paso 6). Si usas `gh`: `gh pr create`.

Opción C · Con Claude Code o Codex (le pides y él hace los comandos)

Ambos son asistentes de IA que corren en tu terminal, leen tu repo y ejecutan los comandos de git por ti. Sirve si no quieres memorizar nada.

Antes de empezar: qué es "la terminal"

Es una ventana negra donde escribes órdenes en vez de hacer clic. No rompe nada por abrirla. Cada vez que esta guía diga "escribe esto", copia el texto, pégalo en la terminal (clic derecho o `Ctrl+V`) y pulsa **Enter**.

Cómo abrirla en Windows: tecla Windows → escribe `PowerShell` → Enter. **En macOS:** `Cmd + Espacio` → escribe `Terminal` → Enter.

Truco para abrirla en la carpeta correcta: abre la carpeta del repo en el Explorador de archivos, haz clic en la barra de dirección de arriba, escribe `powershell` y pulsa Enter. Se abre la terminal ya dentro de esa carpeta.

Una sola vez: instalar lo necesario

Paso 1 · Instalar Git (el programa que guarda las versiones)

1. Descarga desde https://git-scm.com/downloads e instala dando **Next** a todo.
2. Cierra y vuelve a abrir la terminal. Comprueba que quedó bien:

   ```
   git --version
   ```

   Debe responder algo como `git version 2.x.x`.

Paso 2 · Instalar Node.js (lo necesitan los asistentes para funcionar)

1. Descarga la versión **LTS** desde https://nodejs.org e instala dando **Next** a todo.
2. Cierra y vuelve a abrir la terminal. Comprueba:

   ```
   node --version
   ```

   Debe mostrar un número de versión (ej. `v22.x.x`).

Paso 3 · Instalar el asistente (elige **uno**)

Asistente	Comando de instalación	Para abrirlo	Cuenta que necesitas
Claude Code	`npm install -g @anthropic-ai/claude-code`	`claude`	Cuenta de Claude de pago (Pro o superior) o créditos de API
Codex	`npm install -g @openai/codex`	`codex`	Cuenta de ChatGPT (Plus o superior) o API key de OpenAI

Pega el comando de instalación, pulsa Enter y espera a que termine (1–2 minutos, saldrá mucho texto: es normal). Comprueba:

```
claude --version      # o:  codex --version
```

Si responde con un número de versión, ya lo tienes.

> **Si dice "no se reconoce el comando"** o `claude` / `codex` no se encuentra: cierra **todas** las terminales y abre una nueva. Si sigue igual, repite el Paso 2 (Node.js) y reinicia el computador.
>
> **Si en Windows sale un error rojo de "scripts deshabilitados"** (`running scripts is disabled`): ejecuta una vez `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` , responde `S` y vuelve a intentar.

Paso 4 · Conectar GitHub (para que el asistente pueda crear el PR)

1. Instala la CLI de GitHub desde https://cli.github.com e instala dando **Next** a todo. Reabre la terminal.
2. Escribe:

   ```
   gh auth login
   ```

3. Elige con las flechas y Enter: **GitHub.com** → **HTTPS** → **Yes** (autenticar git) → **Login with a web browser**.
4. Copia el código de 8 caracteres que aparece, pulsa Enter, se abre el navegador, pega el código e inicia sesión con **tu** cuenta de GitHub.

Paso 5 · Clonar el repo (si aún no lo tienes; ver Opción B)

```
git clone https://github.com/yetgx/mcdi500-tmdb-grupo4.git
```

Cada vez: abrir el asistente dentro del repo

1. Abre la terminal **dentro de la carpeta del repo** (truco de arriba) o navega con:

```
cd ruta\a\mcdi500-tmdb-grupo4
```

2. Escribe `claude` (o `codex`) y pulsa Enter.
3. **Primera vez:** te pide iniciar sesión. Se abre el navegador, entras con tu cuenta y vuelves a la terminal.
4. Te preguntará si **confías en esa carpeta**: responde que sí (es tu repo).
5. Verás una caja donde escribir. Ahí pegas el prompt de abajo y pulsas Enter.
6. Cuando te pida permiso para ejecutar algo, lee y elige **Yes** / **Aprobar**. Para salir: escribe `/exit` o pulsa `Ctrl+C` dos veces.

Cada vez: pega este prompt

```
Voy a subir mis cambios al repo siguiendo docs/GUIA_SUBIR_CAMBIOS.md.
1. Actualiza main con git pull.
2. Crea una rama nueva con nombre tipo/descripcion (feat, fix, data o docs).
3. Muéstrame git status y haz commit SOLO de estos archivos: <lista de archivos>.
   Mensaje con prefijo feat/fix/data/docs, en imperativo.
4. Haz push de la rama y abre un Pull Request con gh, rellenando la plantilla
   de .github/pull_request_template.md.
Nunca hagas push a main, nunca uses --force ni reset --hard, y no toques data/raw/.
Pídeme confirmación antes de commit y antes de push.
```

Cuidados importantes

- **Lee lo que te pide aprobar.** Antes de aceptar un commit o push, revisa qué archivos incluye. Si aparece `.venv/`, `.env` o algo de `data/raw/`, di que no.
- **Dile siempre qué archivos subir.** Si dices "sube todo", puede arrastrar basura.
- **Conflictos en notebooks:** pídele que te explique el conflicto, pero avisa al grupo antes de resolverlo.
- **Nunca pegues claves ni datos personales** en el chat con el asistente.
- El asistente puede equivocarse: el **PR revisado por un compañero** sigue siendo la red de seguridad. No lo saltes.
- Para revisar un PR ajeno también sirve: `"Revisa el PR #3 y dime si cumple el checklist"`.

Paso 6 · El Pull Request, explicado

1. Rellena la plantilla (ya viene sola): **Qué cambia**, **Por qué**, **Checklist**.
2. Los demás integrantes se piden como revisores automáticamente.
3. Copia el enlace del PR y pégalo en el WhatsApp del grupo: "*PR para revisar: — necesito 1 aprobación*".
4. Cuando alguien aprueba, pulsa **Merge pull request**.
5. Borra la rama (botón **Delete branch** que sale después). Para la siguiente tarea, vuelve al paso 1.

Si te piden cambios en la revisión

No abras otro PR. Haz los cambios en tu misma rama, commit y push: el PR se actualiza solo.

Checklist antes de abrir el PR

- [ ] El notebook corre completo con **Kernel → Restart Kernel and Run All** sin errores
- [ ] Rutas relativas (nada de `C:\Users\...`) y semilla `42`
- [ ] **No modifiqué** `data/raw/` (los datos originales son intocables)
- [ ] No subí `.venv/`, `.env` ni archivos temporales (el `.gitignore` ya los bloquea; si aparecen en la lista de cambios, no los marques)
- [ ] Mi commit empieza con `docs`, `data`, `feat` o `fix`
- [ ] Actualicé `README.md` o `docs/bitacora.md` si cambió algo del flujo

Cómo nombrar las cosas

Ramas: `tipo/descripcion-corta`, en minúsculas y con guiones. `feat/f2-imputacion` · `docs/bitacora` · `fix/ruta-datos`

Commits: `prefijo: lo que hiciste en imperativo y específico`

Prefijo	Úsalo para	Ejemplo
`feat`	Análisis o código nuevo	`feat: agrega grafico de presupuesto vs recaudacion`
`fix`	Corregir un error	`fix: corrige ruta relativa al leer movies_raw.csv`
`data`	Datos procesados	`data: guarda dataset limpio en processed`
`docs`	README, bitácora, diccionario	`docs: registra decision de imputacion en bitacora`

Mal: `cambios`, `listo`, `arreglo final v2`. Bien: lo que otro entiende sin abrir el archivo.

Problemas típicos

"Mi rama quedó desactualizada / el PR dice que hay conflictos" Alguien más mergeó antes que tú. En GitHub Desktop: `Branch → Update from main`. En terminal: `git switch main && git pull && git switch tu-rama && git merge main`. Si hay conflicto en un notebook, avisa al grupo antes de resolver a ciegas: los `.ipynb` son JSON y se rompen fácil. Lo más seguro es que solo una persona toque un notebook a la vez.

"Commiteé en `main` por error" No pasa nada, GitHub no te dejará hacer push (está protegida). Crea la rama desde ahí: `git switch -c mi-rama` y haz push de esa; después `main` local se arregla con `git fetch && git switch main && git reset --hard origin/main`. Si dudas, pregunta antes del `reset`.

"Me pide usuario y contraseña" GitHub ya no acepta contraseña. Usa GitHub Desktop (maneja el login solo) o inicia sesión con `gh auth login`.

"Aparecen cientos de archivos cambiados" Probablemente agregaste `.venv/` o cambió el formato de saltos de línea. Desmarca todo lo que no sea tuyo. Si pasa en un `.csv` de `data/raw/`, **descártalo** (`Discard changes`): no debe cambiar.

"Subí algo que no debía (datos personales, claves)" Avisa al grupo ya. Borrarlo en un commit nuevo no lo elimina del historial.

"Me trabé y no sé qué hacer" No improvises con `reset --hard` ni `push --force`. Escribe al grupo con una captura del mensaje de error.

Chuleta rápida

```
git switch main && git pull        # empezar al día
git switch -c feat/lo-que-sea      # rama nueva
git status                         # ¿qué cambió?
git add archivo                    # preparar
git commit -m "feat: ..."          # guardar
git push -u origin feat/lo-que-sea # subir
```